Building Applications on Azure

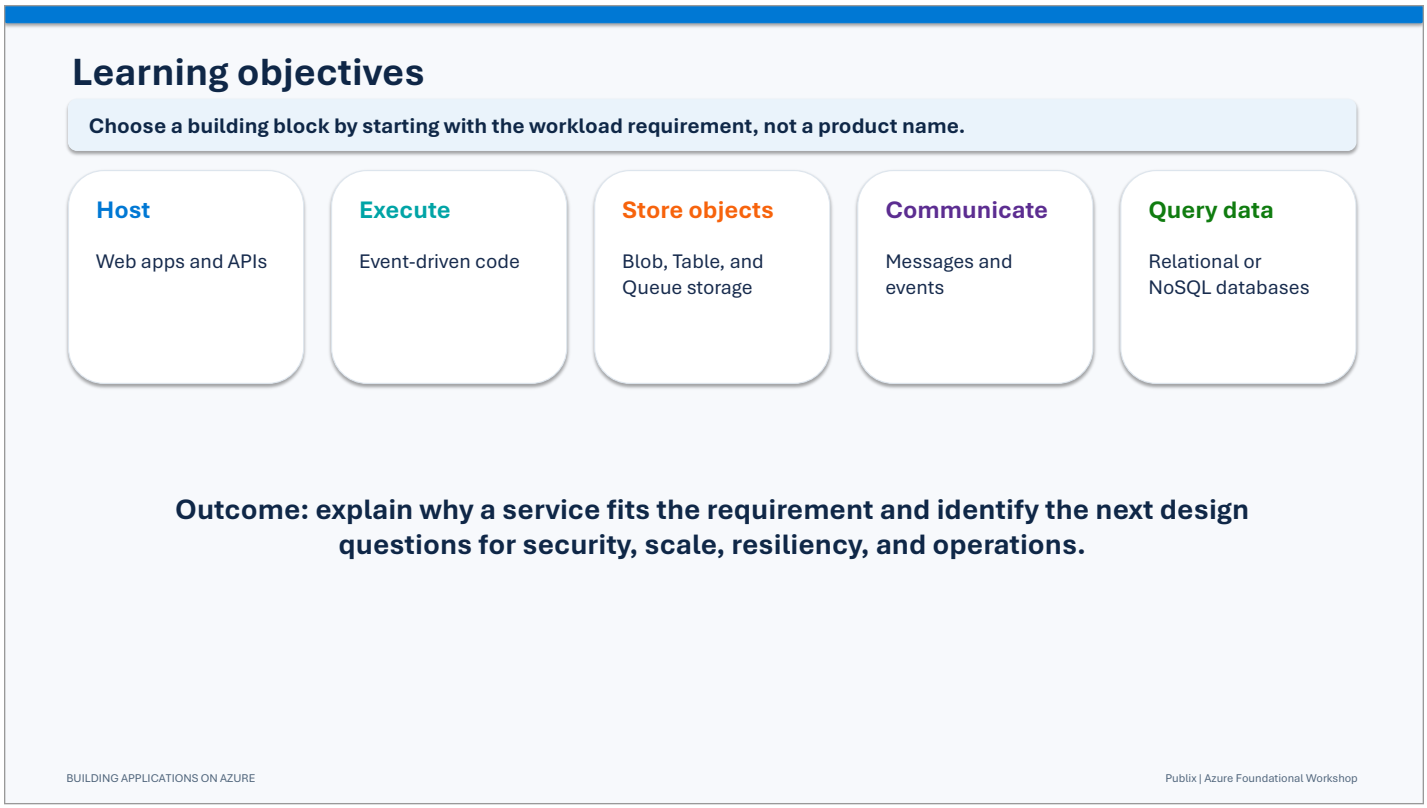
Selecting managed services for hosting, data, messaging, and databases

Level 100–200 | Services selected for use at Publix

Publix Azure Foundational Workshop

Purpose: introduce the managed services Publix selected for application hosting, data, messaging, and databases, and a simple way to choose among them. Frame the module around one question: what does the workload need? Product names come second. The lab at the end builds a small order path that attendees operate in the next module, so point out each service as it appears.

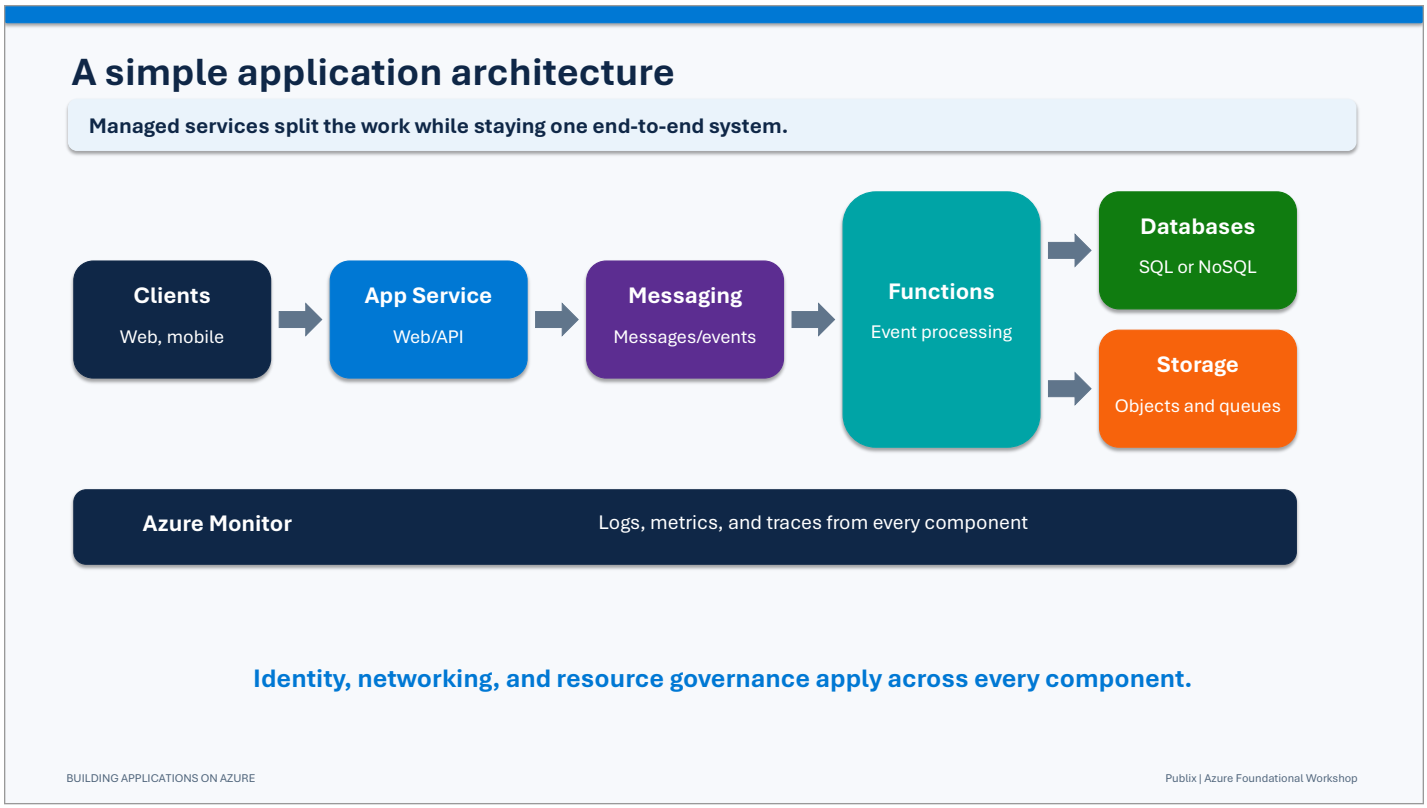
[Sources]
- https://learn.microsoft.com/azure/architecture/guide/technology-choices/



Slide 2 of 18

Walk the five building blocks left to right: host web apps and APIs, run event-driven code, store objects, move messages and events, and query structured data. Stress the outcome: by the end, attendees should be able to explain why a service fits a requirement and name the next questions to ask about security, scale, resiliency, and operations. They aren't expected to memorize features.

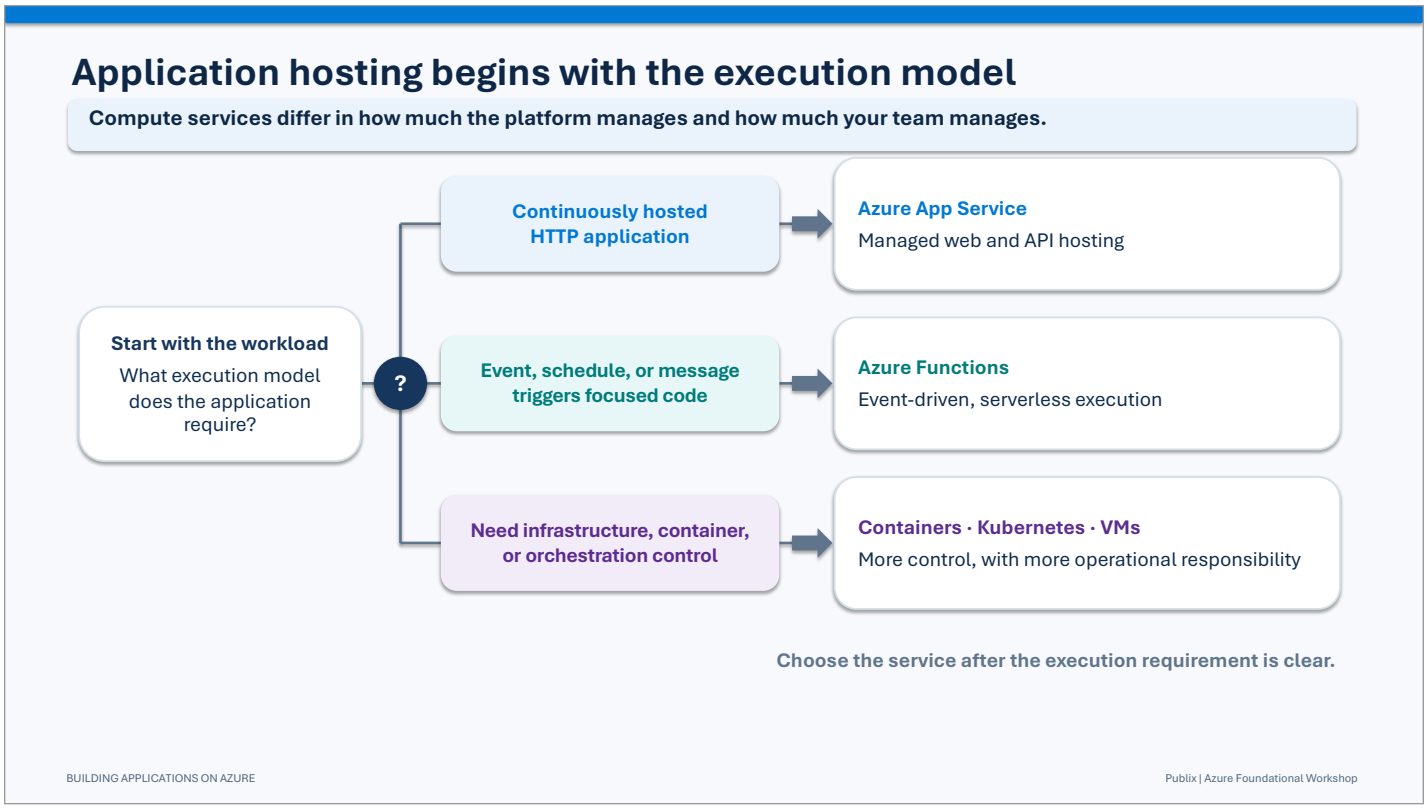
[Sources]
- https://learn.microsoft.com/azure/architecture/guide/technology-choices/



Slide 3 of 18

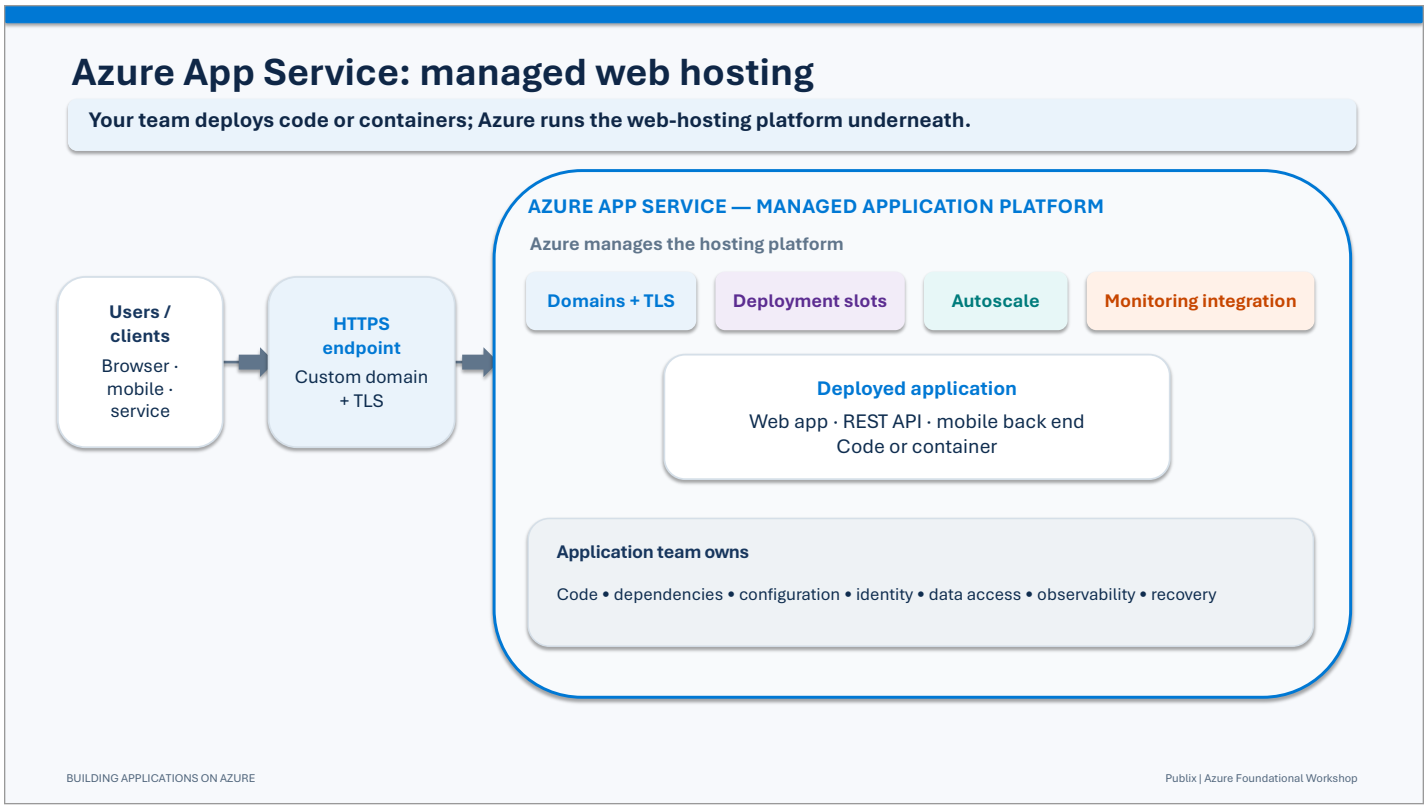
Follow one request left to right. A client calls an API hosted in App Service. The API hands work to a queue or topic and returns quickly. Functions picks up the work, then writes to a database or to Storage. Azure Monitor collects logs and metrics from every component. Point out that identity, networking, and governance from the first module apply to each box. This same shape returns in the lab and in the operations module.

[Sources]
- https://learn.microsoft.com/azure/architecture/guide/



Start from how the code runs, not from a product name. A continuously hosted HTTP application fits App Service. Code that runs when an event, a schedule, or a message arrives fits Functions. Teams that need control of the operating system, containers, or orchestration move toward containers, Kubernetes, or VMs, and take on more operational work. Ask the room where their current applications would land.

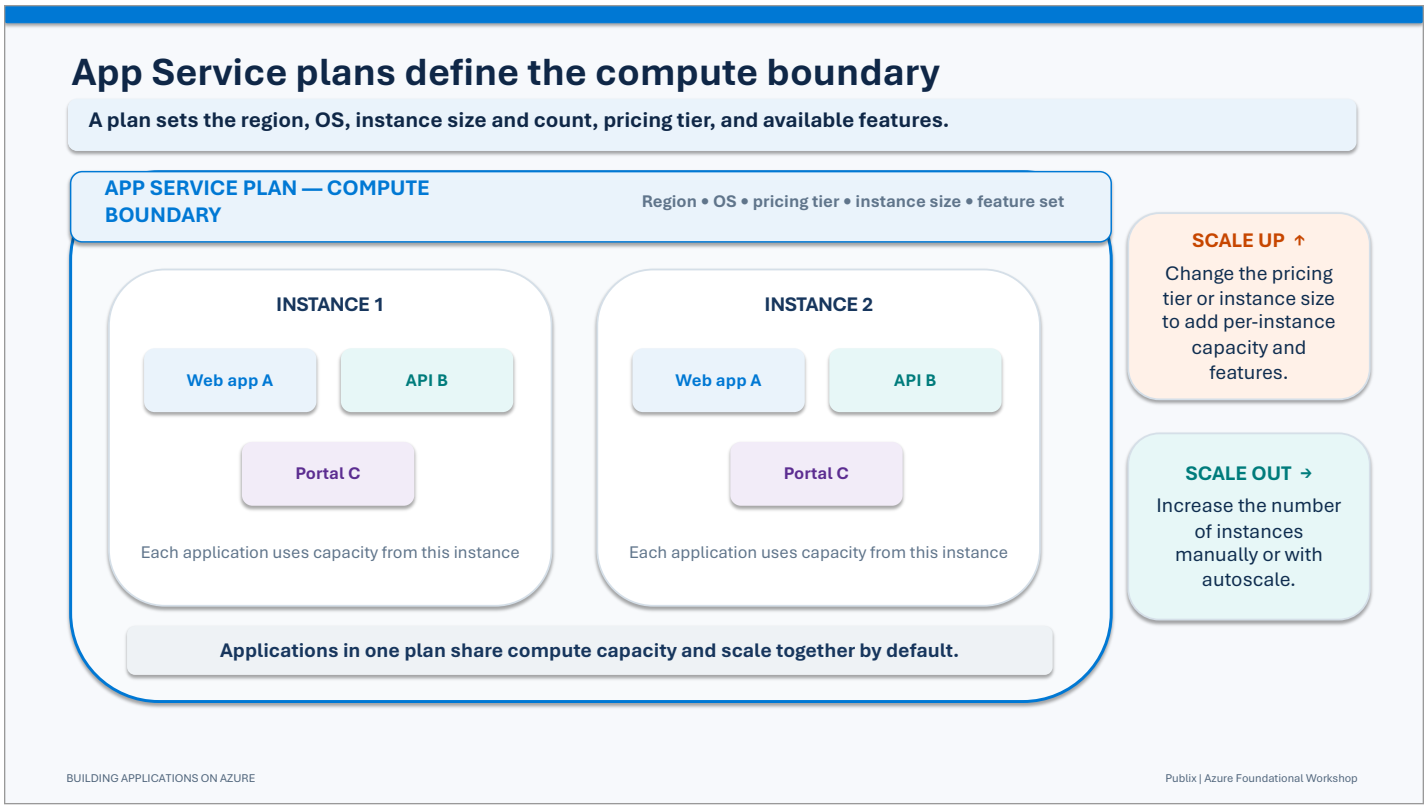
[Sources]
- https://learn.microsoft.com/azure/app-service/overview
- https://learn.microsoft.com/azure/azure-functions/functions-overview



Slide 5 of 18

App Service is a managed platform for web apps and APIs. Azure runs the servers, patching, and load balancing; the team brings code or a container. Call out the platform features attendees get without building them: custom domains with TLS, deployment slots for safer releases, autoscale, and monitoring integration. Then the bottom row: the team still owns its code, dependencies, configuration, identity, data access, observability, and recovery.

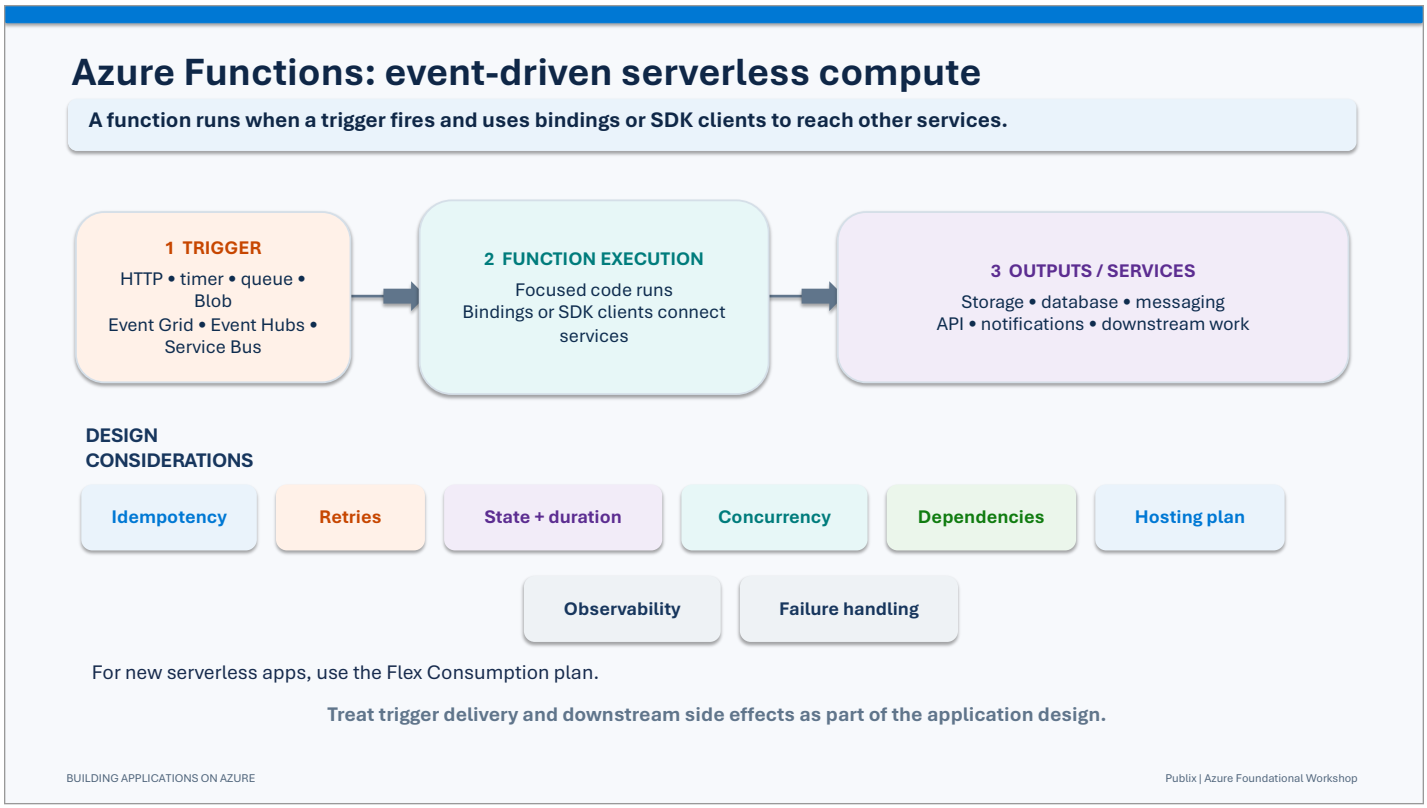
[Sources]
- https://learn.microsoft.com/azure/app-service/overview



Slide 6 of 18

The plan is the compute. It sets the region, operating system, pricing tier, instance size, and instance count. Every app in a plan runs on every instance in that plan, so the apps share capacity and scale together. Scale up for bigger instances or tier features; scale out for more instances, manually or with autoscale. A common practice is to put noisy or critical apps in their own plans.

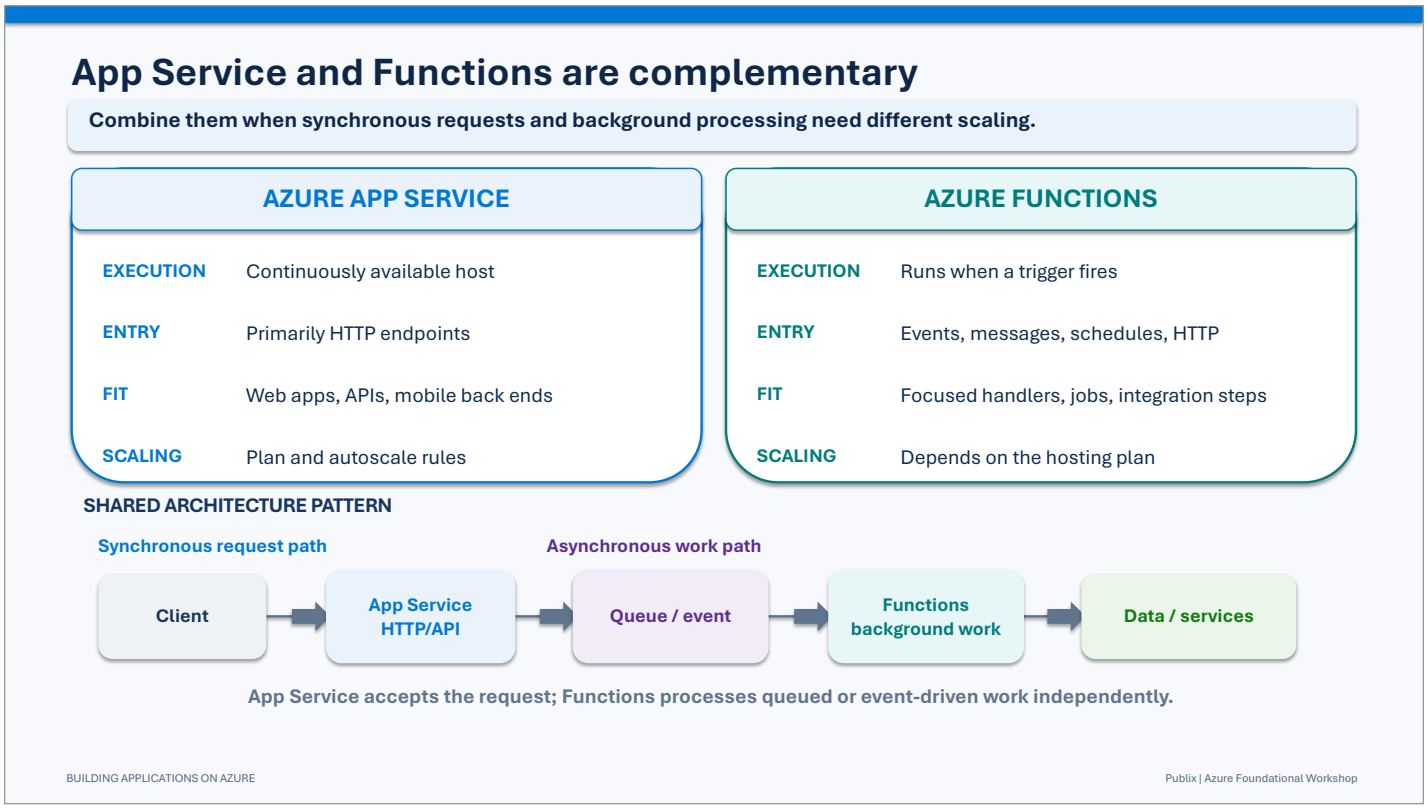
[Sources]
- https://learn.microsoft.com/azure/app-service/overview-hosting-plans



Slide 7 of 18

A function runs when a trigger fires: an HTTP call, a timer, a queue message, or an event. Bindings and SDK clients connect it to other services with little code. For new serverless apps, the workshop uses the Flex Consumption plan; the older Linux Consumption plan retires in September 2028. Spend time on the design considerations: a trigger can deliver the same message more than once, so handlers must be idempotent; retries and failure handling need a plan; and long-running state needs a different pattern.

[Sources]
- https://learn.microsoft.com/azure/azure-functions/functions-overview
- https://learn.microsoft.com/azure/azure-functions/functions-triggers-bindings
- https://learn.microsoft.com/en-us/azure/azure-functions/flex-consumption-plan



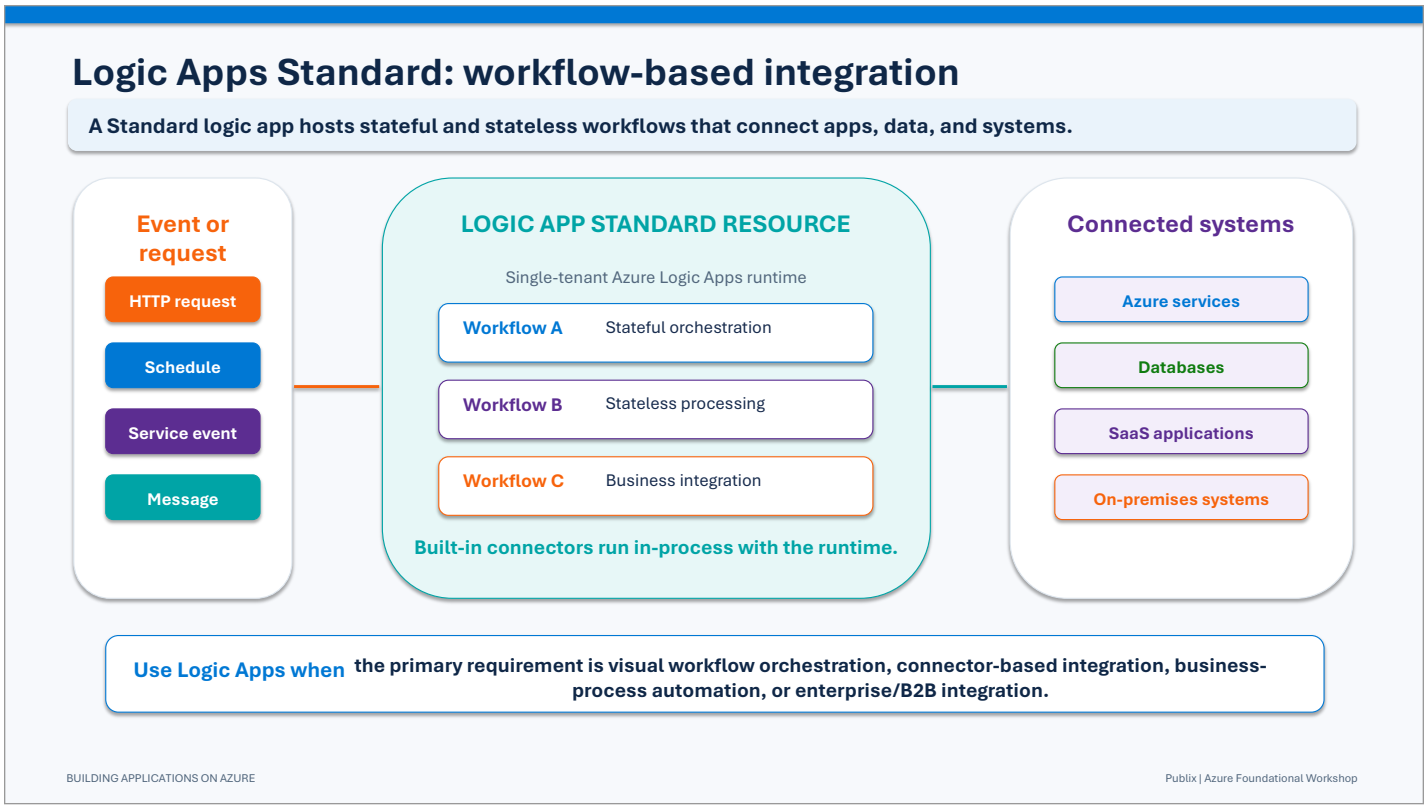
Slide 8 of 18

App Service and Functions often handle two halves of the same application. App Service keeps an API available for synchronous requests. Functions runs only when work arrives and scales with the queue. The pattern at the bottom is the one to remember: the API accepts the request and puts the work on a queue, then Functions processes it independently. That keeps user response times steady even when downstream work is slow or failing. Lab 2 builds this exact pattern.

[Sources]
- https://learn.microsoft.com/azure/app-service/overview
- https://learn.microsoft.com/azure/azure-functions/functions-overview

Demonstration

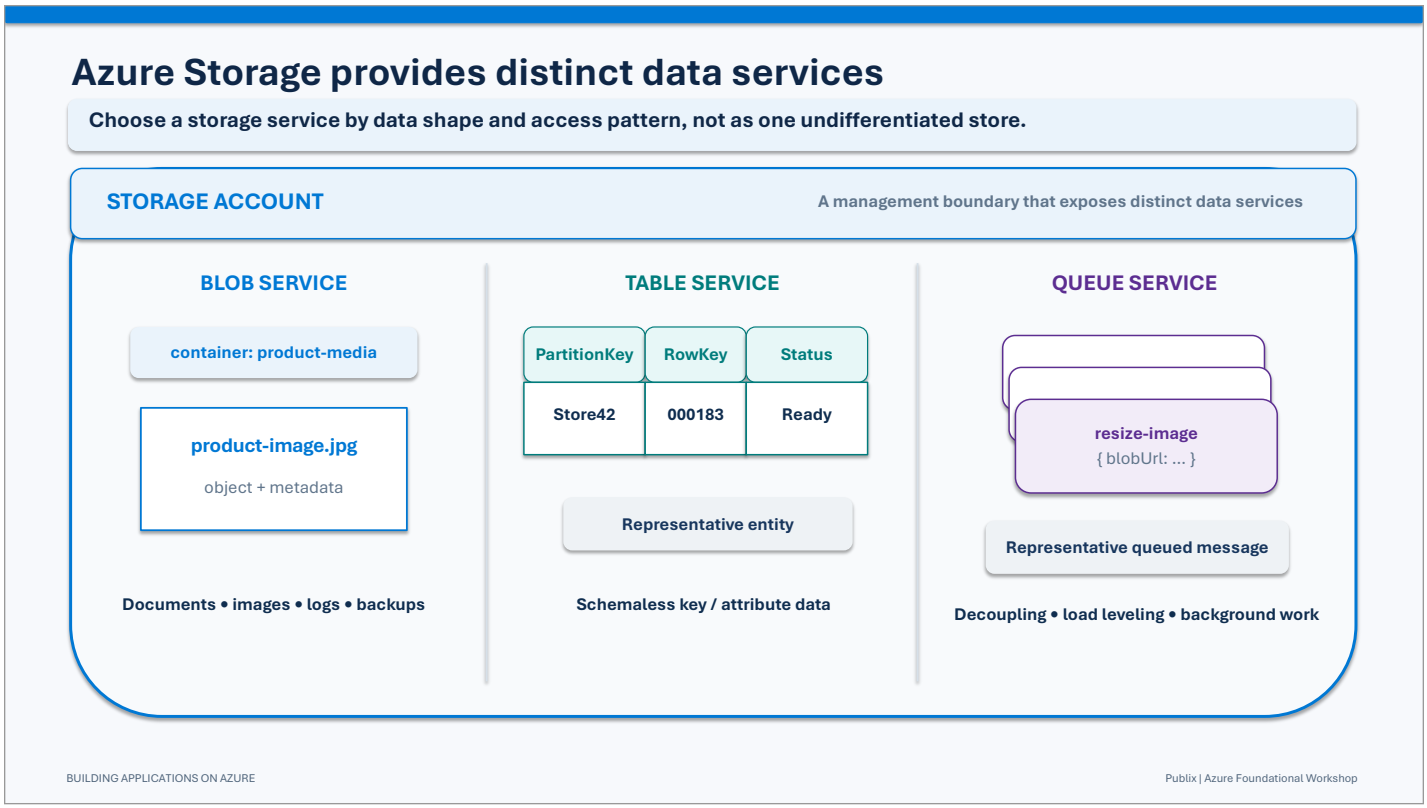
See notes



Slide 10 of 18

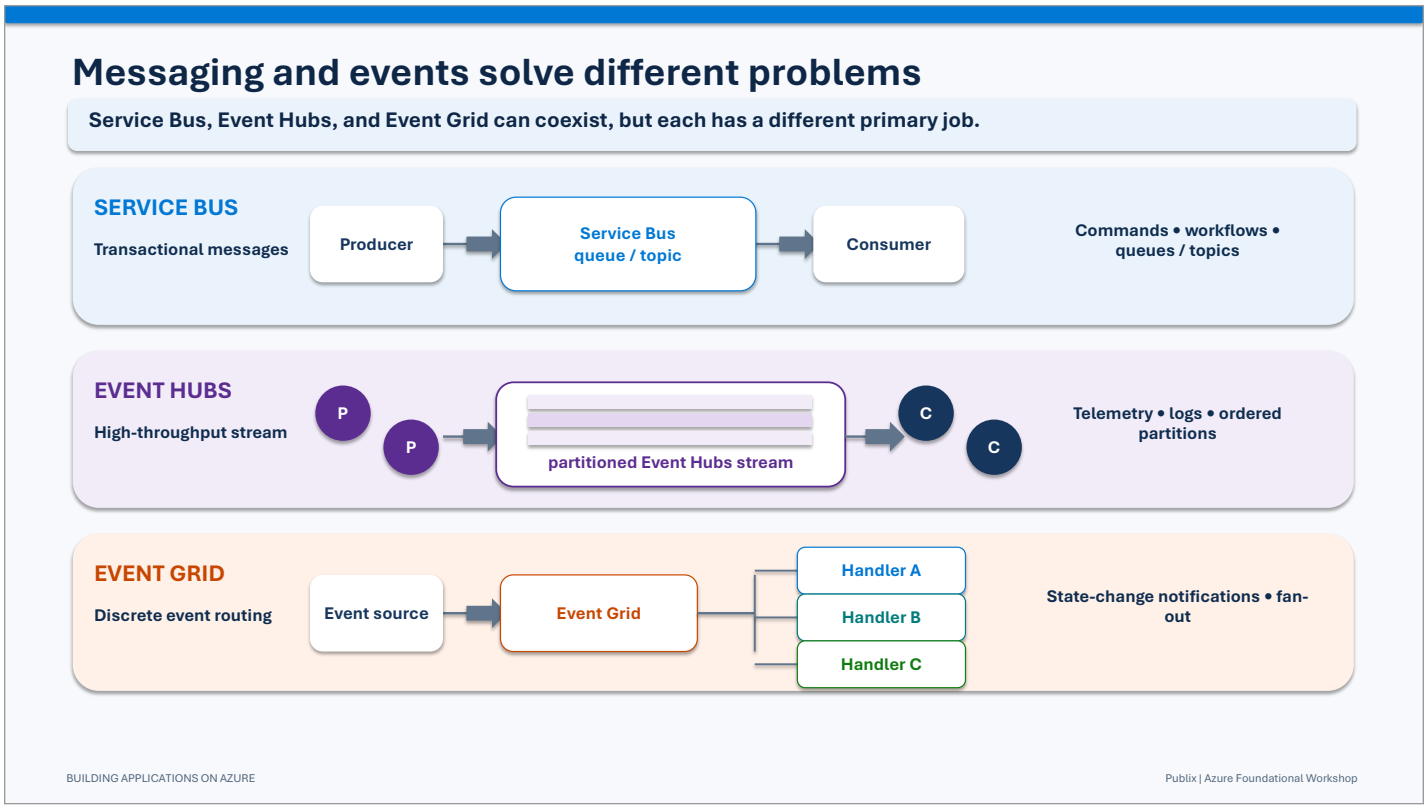
High-level concept: Azure Logic Apps is a cloud platform for creating automated workflows that integrate applications, data, services, and systems. A Standard logic app resource can contain one or multiple stateful or stateless workflows that run in single-tenant Azure Logic Apps or an App Service Environment v3. Connectors provide prebuilt triggers and actions. Built-in connectors run natively in the Azure Logic Apps runtime; managed connectors provide Microsoft-managed access to service and system APIs. Logic Apps Standard is a strong fit when the workload centers on workflow orchestration and integration rather than primarily hosting an HTTP application or writing focused event-handler code. Keep this slide brief; the separate two-day Logic Apps workshop covers it in depth.

[Sources]
- https://learn.microsoft.com/azure/logic-apps/logic-apps-overview
- https://learn.microsoft.com/azure/logic-apps/single-tenant-overview-compare
- https://learn.microsoft.com/azure/connectors/introduction
- https://learn.microsoft.com/azure/connectors/built-in



One storage account exposes several services, so choose by the shape of the data and how it's accessed. Blob Storage holds objects such as documents, images, logs, and backups. Table Storage holds simple key-and-attribute entities. Queue Storage holds small messages for background work. The partition key and row key on the slide illustrate the idea; they aren't a lesson in table design, so keep the discussion at the level of which service fits.

[Sources]
- https://learn.microsoft.com/azure/storage/common/storage-introduction
- https://learn.microsoft.com/azure/storage/blobs/storage-blobs-introduction
- https://learn.microsoft.com/azure/storage/tables/table-storage-overview
- https://learn.microsoft.com/azure/storage/queues/storage-queues-introduction



Slide 12 of 18

These three services can work together, but each has a primary job. Service Bus carries business messages and commands that must not be lost, with queues, topics, ordering, and dead-lettering. Event Hubs ingests high-volume streams, such as telemetry and logs, in partitions. Event Grid routes discrete events, such as 'a blob was created', to the handlers that react to them. A quick test for any new requirement: is it a command that needs a guaranteed handler, a stream to analyze, or a notification that something happened?

[Sources]
- https://learn.microsoft.com/azure/service-bus-messaging/compare-messaging-services

Events vs Messages



Events

Lightweight notification of condition or state change

Publisher not concerned with how it is handled

May be discreet or part of a sequenced series

Events have information about the event but not the data itself



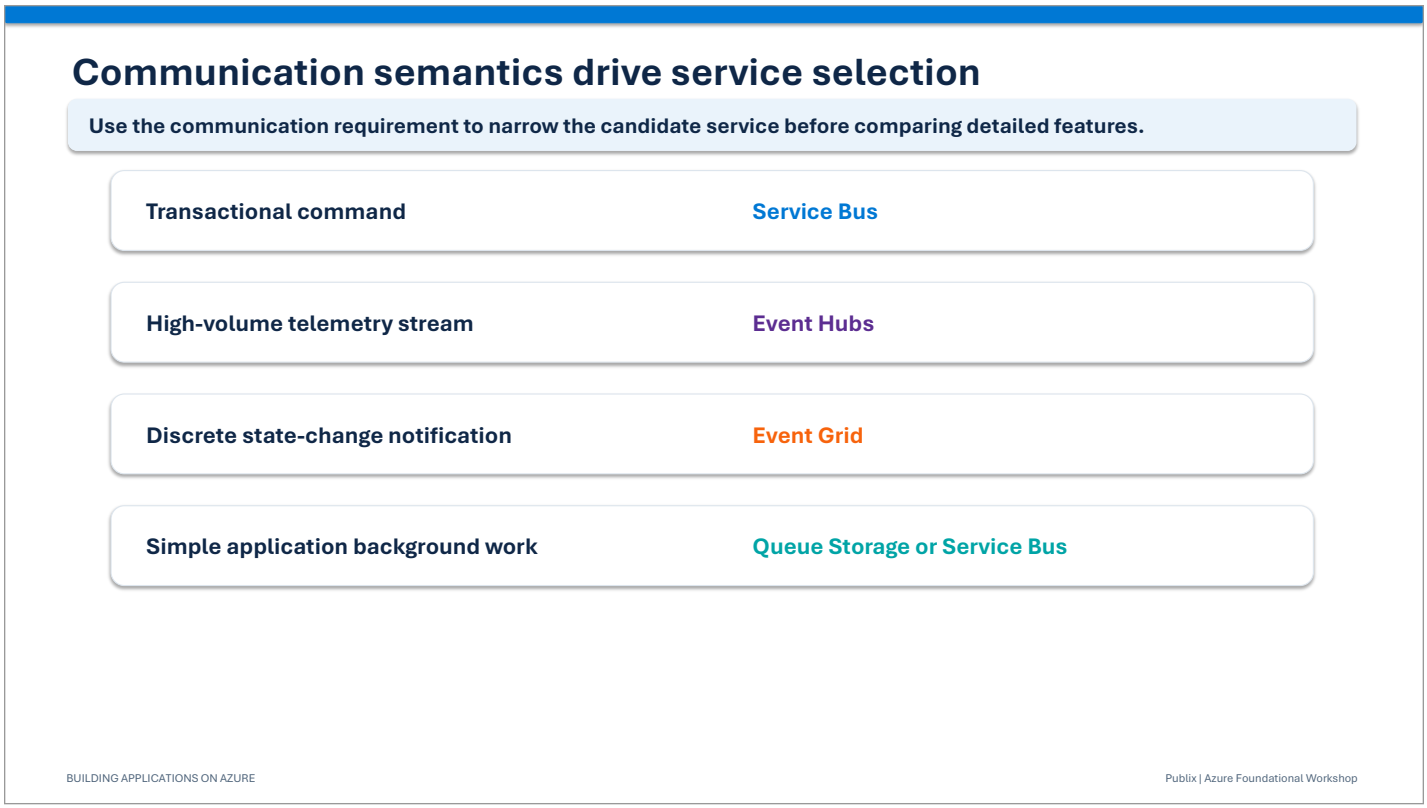
Messages

Raw data produced by a service

Consumed elsewhere

Publisher is concerned about how message is processed

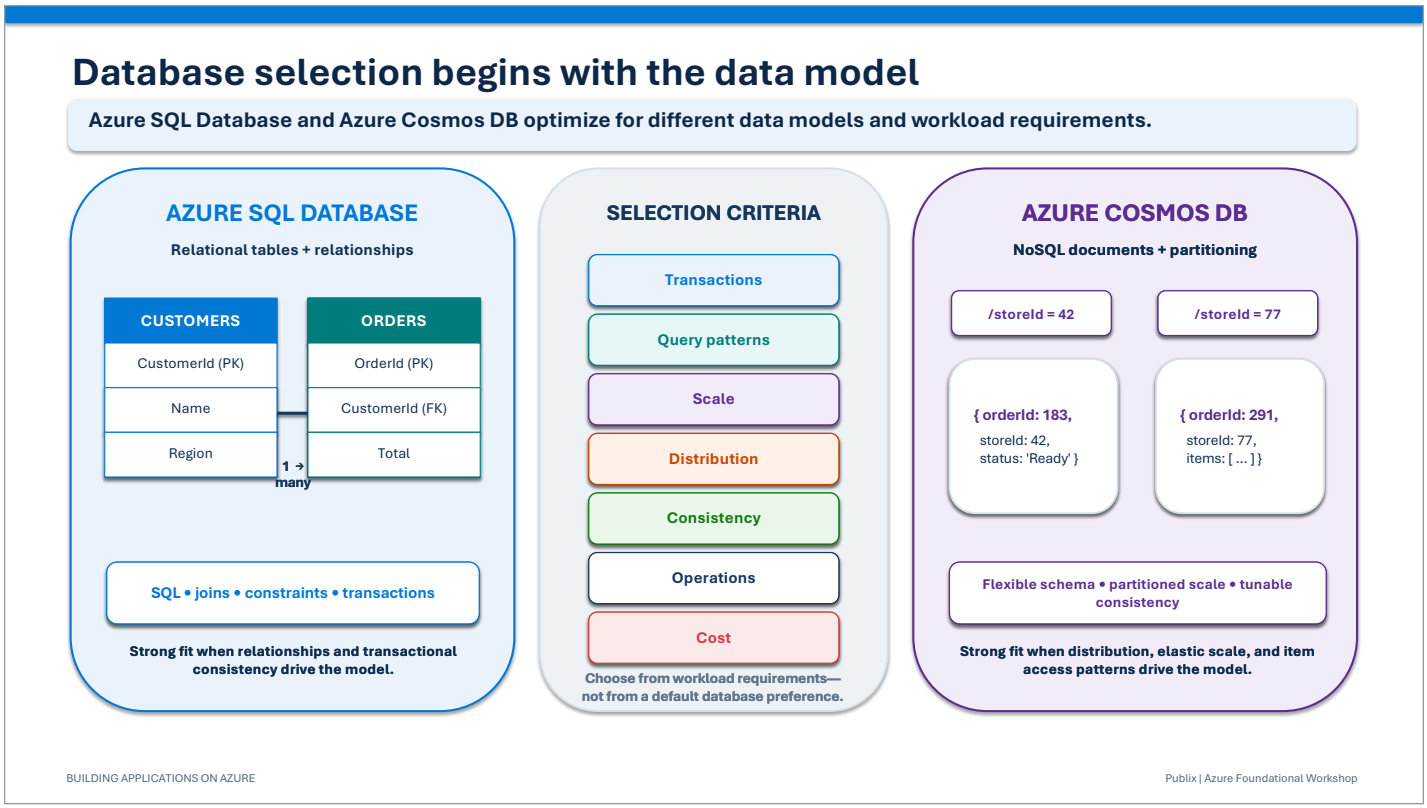
(No speaker notes)



Slide 14 of 18

These are starting points, not complete designs.

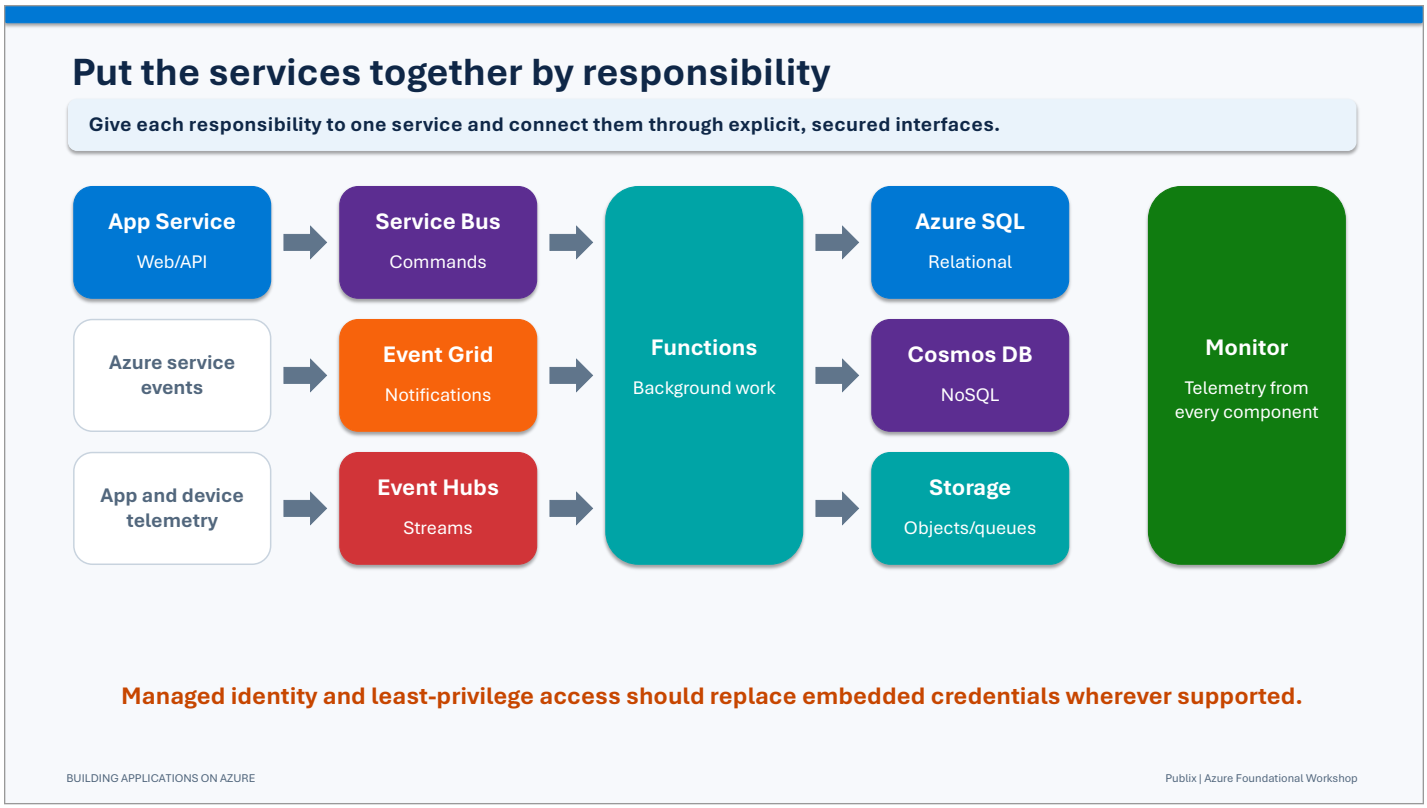
[Sources]
- https://learn.microsoft.com/azure/service-bus-messaging/compare-messaging-services



Slide 15 of 18

Azure SQL Database and Azure Cosmos DB optimize for different data models and workload requirements.

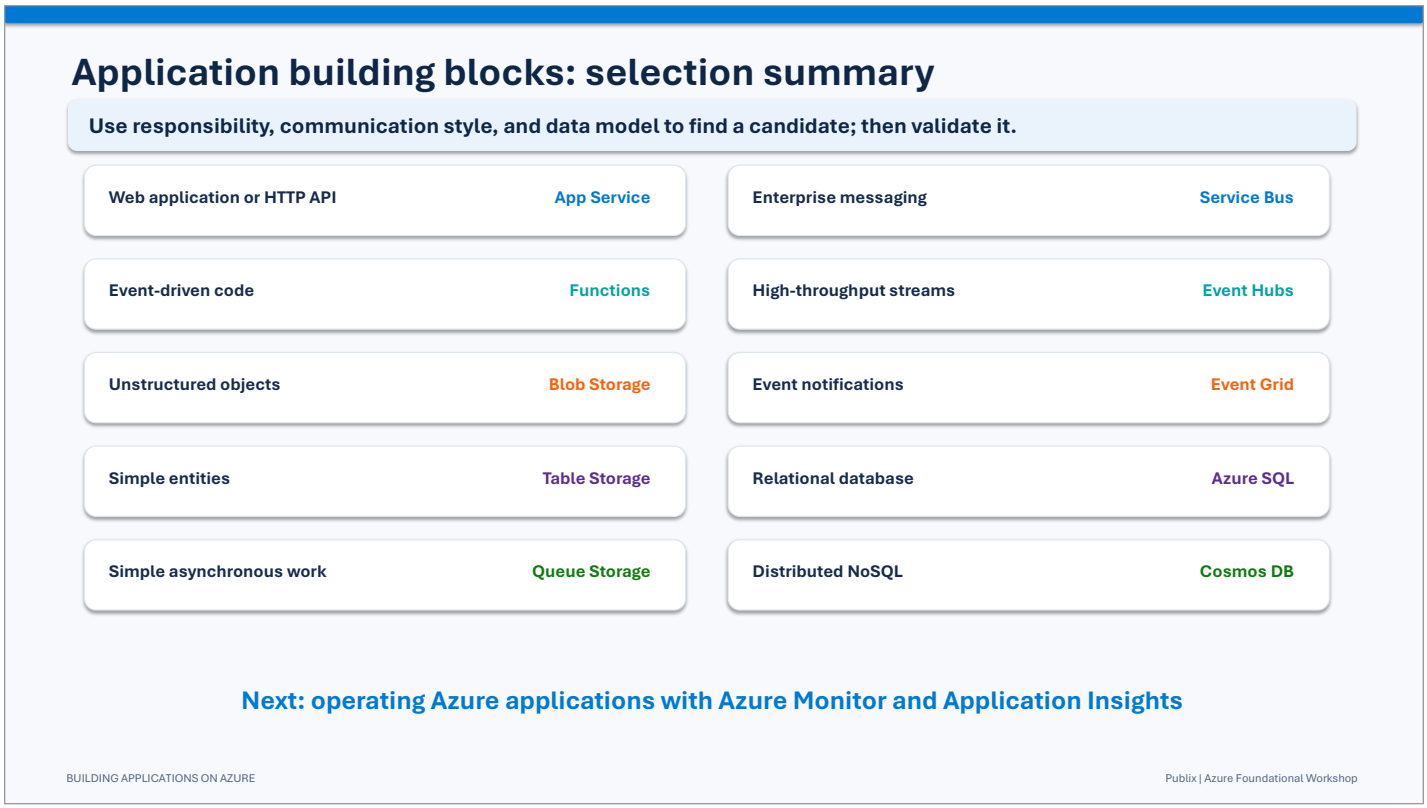
[Sources]
- https://learn.microsoft.com/azure/azure-sql/database/sql-database-paas-overview
- https://learn.microsoft.com/azure/cosmos-db/introduction



Slide 16 of 18

Read the diagram by responsibility. App Service accepts requests and sends commands through Service Bus. Event Grid delivers events raised by Azure services, and Event Hubs ingests telemetry streams from apps and devices. Functions does the background work and writes to the right store: Azure SQL for relational data, Cosmos DB for distributed NoSQL data, and Storage for objects. Azure Monitor collects telemetry from all of it. Every arrow is a place to use managed identity and least-privilege access instead of embedded credentials. The diagram is illustrative, not prescriptive.

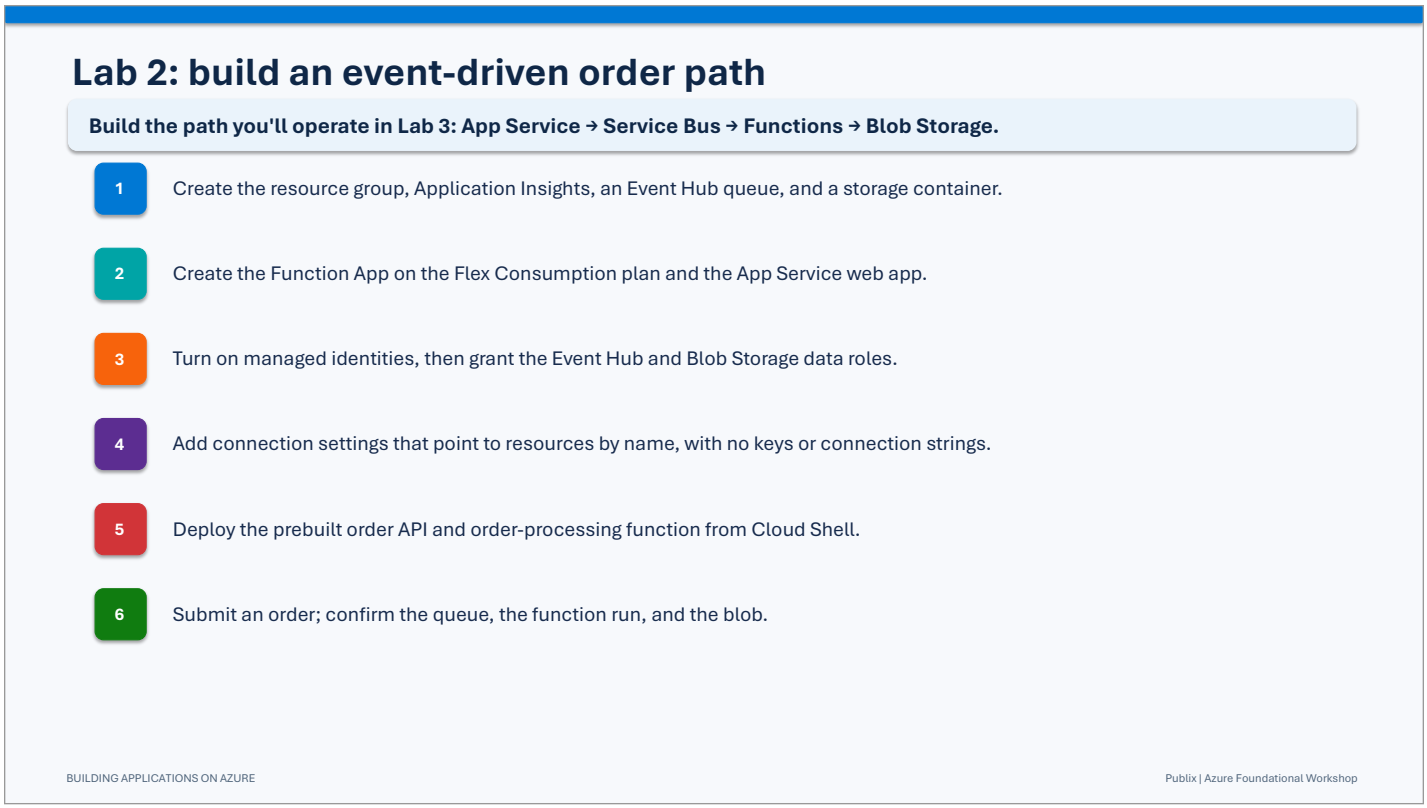
[Sources]
- https://learn.microsoft.com/azure/architecture/guide/



Slide 17 of 18

Use this as the take-away card. Start from the responsibility, the communication style, and the data model to pick a candidate service, then confirm security, scale, resiliency, and operations. One nuance from messaging: Queue Storage is enough for simple background work; choose Service Bus when you need ordering, sessions, dead-lettering, or transactions. Next, the operations module shows how to see what these services are doing in production.

[Sources]
- https://learn.microsoft.com/azure/architecture/guide/technology-choices/
- https://learn.microsoft.com/azure/service-bus-messaging/compare-messaging-services



Slide 18 of 18

Lab guide: Building Applications on Azure - Lab 2. Attendees build the order path used in Lab 3: an App Service web app accepts orders and sends them to a Service Bus queue, and a function on the Flex Consumption plan processes each order and writes it to Blob Storage. Prebuilt code is provided, so no coding is required. Identity-based connections replace keys and connection strings. Resources stay in place for Lab 3, which ends with cleanup.

[Sources]
- https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-quickstart-portal
- https://learn.microsoft.com/en-us/azure/azure-functions/flex-consumption-plan
- https://learn.microsoft.com/en-us/azure/azure-functions/functions-identity-based-connections-tutorial-2
- https://learn.microsoft.com/en-us/azure/app-service/quickstart-python
- https://learn.microsoft.com/azure/azure-functions/functions-monitoring